4.1. Multi Thread

program thread01.java
program thread01.java berikut merupakan contoh program multithread yang menggunakan ***implements Runnable***

```
class threadKu implements Runnable {
  private int countDown = 5;
  private static int threadCount = 0;
  private int threadNumber = ++threadCount;

  public threadKu() {
    System.out.println("Pembuatan Thread ke " +
        threadNumber);
  }
  public void run() {
    while(true) {
      System.out.println("Thread ke " +
          threadNumber + "(" + countDown + ")");
      if(--countDown == 0) return;
   }
  }
}

public class thread01 {
  public static void main(String[] args) {
    for(int i = 0; i < 6; i++) {
      threadKu o = new threadKu();
      Thread oTh = new Thread(o);
      oTh.start();
    }
    System.out.println("Semua Thread telah dibuat!");
  }
}
```

program thread02.java
program thread02.java berikut merupakan contoh program multithread yang menggunakan ***extends Thread***

```
class myThread2 extends Thread {
  private int countDown = 5;
  private static int threadCount = 0;
  private int threadNumber = ++threadCount;
  public myThread2() {
    System.out.println("Pembuatan thread ke " +
              threadNumber);
```

```
  }
  public void run() {
    while(true) {
      System.out.println("Thread ke " +
          threadNumber + " (" + countDown + ")");
      if(--countDown == 0) return;
    }
  }
}

public class thread02 {
  public static void main(String[] args) {
    for(int i = 0; i < 5; i++)
      new myThread2().start();
    System.out.println("Semua Threads telah dibuat dan dijalankan!");
  }
}
```

Jelaskan perbedaan dari thread01.java dan thread02.java pada laporan

thread03.java

```
class threadku03 extends Thread{
     private static int i=0;
     public void run(){
          while(true) i++;
     }
     public int getI() {return i;}
}

public class thread03{
     public static void main(String[] args){
          boolean tunda = false;
          threadku03 trd3 = new threadku03();
          trd3.start();
          for(int j=0;j<100;j++){
               if (j%25 == 0){
                    if(tunda){
                         System.out.println("Sebeleum resume j = "+j+", i =
"+trd3.getI());
                         trd3.resume();
                         System.out.println("Setelah resume j = "+j+", i =
"+trd3.getI());
                    } else {
                         System.out.println("Sebeleum suspend j = "+j+", i =
"+trd3.getI());
                         trd3.suspend();
                         System.out.println("Setelah suspend j = "+j+", i =
```

```
"+trd3.getI());
                        }
                        tunda = !tunda;
                    }
                }
                trd3.stop();
        }
}
```

4.2. Perkalian Matrix

Kerjakan dan jalankan Program Perkalian Matriks seperti yang ada pada slide

4.3. Fork-Join Parallelism

```
import java.util.concurrent.*;

public class SumTask extends RecursiveTask<Integer>
{
   static final int THRESHOLD = 1000;

   private int begin;
   private int end;
   private int[] array;

   public SumTask(int begin, int end, int[] array) {
      this.begin = begin;
      this.end = end;
      this.array = array;
   }

   protected Integer compute() {
      if (end - begin < THRESHOLD) {
         int sum = 0;
         for (int i = begin; i <= end; i++)
            sum += array[i];

         return sum;
      }
      else {
         int mid = (begin + end) / 2;

         SumTask leftTask = new SumTask(begin, mid, array);
         SumTask rightTask = new SumTask(mid + 1, end, array);

         leftTask.fork();
         rightTask.fork();

         return rightTask.join() + leftTask.join();
      }
   }
}
```